

Mémento Python pour l'ACP (Analyse en Composantes Principales)

1. BIBLIOTHÈQUES ESSENTIELLES

```
python

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
```

2. PANDAS - Manipulation de données

Création de DataFrames

```
python

df = pd.DataFrame(data, index=liste_noms)
# data : dictionnaire {colonne: [valeurs]}
# index : noms des lignes
```

Accès aux données

```
python

df.shape      # (nb_lignes, nb_colonnes) → tuple
df.shape[0]   # Nombre de lignes
df.shape[1]   # Nombre de colonnes
df.index      # Liste des noms de lignes
df.columns    # Liste des noms de colonnes
df.loc['nom']  # Accès par nom de ligne
df['colonne']  # Accès à une colonne
```

Statistiques descriptives

```
python
```

```
df.mean()           # Moyenne de chaque colonne
df.std(ddof=0)       # Écart-type (ddof=0 pour population, ddof=1 pour échantillon)
df.describe()        # Statistiques complètes (count, mean, std, min, 25%, 50%, 75%, max)
df.describe().loc[['mean', 'std']] # Extrait lignes spécifiques
```

Tri et affichage

```
python

df.sort_values(by='colonne', ascending=False) # Tri décroissant
df.head(n)           # Affiche les n premières lignes
display(df)          # Affichage formaté dans Jupyter
```

3. NUMPY - Calculs numériques

Opérations sur arrays

```
python

np.array([1, 2, 3])   # Création d'un array
np.arange(start, stop) # Crée [start, start+1, ..., stop-1]
np.sum(array)         # Somme de tous les éléments
np.round(array, decimales) # Arrondi
np.sqrt(array)        # Racine carrée élément par élément
```

Algèbre linéaire

```
python

np.dot(A, B)          # Produit matriciel  $A \times B$ 
A.T                   # Transposée de la matrice A
np.linalg.eig(matrice) # Retourne (valeurs_propres, vecteurs_propres)
```

Tri et indexation

```
python
```

```
np.argsort(array)      # Indices qui trieraient le tableau (ordre croissant)
array[::-1]           # Inverse l'ordre des éléments
array[:n]              # Conserve les n premiers éléments
array[:, :n]           # Conserve les n premières colonnes (matrices)
array[indices]         # Réordonne selon une liste d'indices
```

4. MATPLOTLIB - Visualisations de base

Configuration

```
python

plt.rcParams['figure.figsize'] = (largeur, hauteur) # Taille par défaut
```

Types de graphiques

```
python

plt.figure(figsize=(largeur, hauteur)) # Crée une nouvelle figure
plt.plot(x, y, marker='o', linestyle='--', label='nom') # Courbe
plt.scatter(x, y, c='couleur', alpha=transparence)    # Nuage de points
```

Annotations et axes

```
python

plt.text(x, y, 'texte', fontsize=taille, ha='center', va='center')
# ha : alignement horizontal ('left', 'center', 'right')
# va : alignement vertical ('top', 'center', 'bottom')

plt.arrow(x_départ, y_départ, dx, dy, head_width=largeur, fc='couleur', ec='couleur')
# dx, dy : déplacement relatif
# fc : fill color, ec : edge color
```

Lignes de référence

```
python

plt.axhline(y, color='couleur', linestyle=':', label='nom') # Ligne horizontale
plt.axvline(x, color='couleur', linestyle=':', label='nom') # Ligne verticale
```

Axes et titres

```
python

plt.xlabel('nom axe X')
plt.ylabel('nom axe Y')
plt.title('Titre du graphique')
plt.xlim(min, max)      # Limites axe X
plt.ylim(min, max)      # Limites axe Y
plt.legend()             # Affiche la légende
plt.grid()               # Ajoute une grille
plt.show()               # Affiche le graphique
```

Cercles

```
python

circle = plt.Circle((centre_x, centre_y), rayon, color='couleur', fill=False)
ax = plt.gca()           # Get Current Axes
ax.add_artist(circle)     # Ajoute le cercle au graphique
```

5. SEABORN - Visualisations avancées

```
python

sns.set_theme(style="whitegrid") # Applique un thème esthétique

sns.heatmap(matrice, annot=True, cmap='coolwarm', vmin=min, vmax=max)
# annot=True : affiche les valeurs dans les cellules
# cmap : palette de couleurs ('coolwarm', 'viridis', 'Blues', etc.)
# vmin, vmax : valeurs min et max de l'échelle de couleurs
```

6. SCIKIT-LEARN - Machine Learning

Standardisation

```
python
```

```
scaler = StandardScaler()      # Crée l'objet de standardisation
Z = scaler.fit_transform(df)    # Calcule mean/std puis standardise
# Équivalent à :  $Z = (df - df.mean()) / df.std()$ 
```

ACP (PCA)

```
python

pca = PCA(n_components=nb_axes) # Crée l'objet ACP
coords = pca.fit_transform(Z)   # Calcule l'ACP et projette les données

# Attributs utiles après fit :
pca.explained_variance_        # Valeurs propres
pca.explained_variance_ratio_   # Pourcentage d'inertie expliquée
pca.components_                # Vecteurs propres (transposés)
```

7. FORMULES MATHÉMATIQUES DE L'ACP

Centrage-Réduction

```
python

Z = (df - df.mean()) / df.std(ddof=0)
```

Matrice de corrélation

```
python

n = df.shape[0] # Nombre d'individus
R = (1/n) * np.dot(Z.T, Z)
```

Décomposition spectrale

```
python

valeurs_propres, vecteurs_propres = np.linalg.eig(R)
```

Tri décroissant

```
python
```

```
idx = np.argsort(valeurs_propres)[::-1]
valeurs_propres = valeurs_propres[idx]
vecteurs_propres = vecteurs_propres[:, idx]
```

Part d'inertie expliquée

```
python

inertie_ratio = valeurs_propres / np.sum(valeurs_propres)
```

Coordonnées factorielles (projection des individus)

```
python

F = np.dot(Z, vecteurs_propres)
```

Coordonnées des variables sur les axes

```
python

coord_variables = vecteurs_propres * np.sqrt(valeurs_propres)
```

Contribution d'un individu à un axe

```
python

contribution = (coordonnée_factorielle ** 2) / (n * valeur_propre)
```

8. ASTUCES PRATIQUES

Boucles avec indices

```
python

for i, element in enumerate(liste):
    print(f"Indice {i} : {element}")
```

Opérations élément par élément

```
python
```

```
array ** 2          # Carré de chaque élément  
array * scalaire    # Multiplication par un nombre  
matrice[ligne, colonne]  # Accès à un élément
```

String formatting

```
python  
  
f"Texte {variable:.2f}%" # Formate avec 2 décimales
```

Accès sélectif

```
python  
  
df.loc['nom_ligne']          # Ligne par nom  
df.loc[['nom1', 'nom2']]    # Plusieurs lignes  
df[['col1', 'col2']]        # Plusieurs colonnes
```

9. STRUCTURE TYPIQUE D'UNE ACP

```
python
```

1. Chargement et exploration

```
df = pd.DataFrame(data, index=noms)
```

```
n, p = df.shape
```

2. Standardisation

```
Z = (df - df.mean()) / df.std(ddof=0)
```

3. Matrice de corrélation

```
R = (1/n) * np.dot(Z.T, Z)
```

4. Valeurs et vecteurs propres

```
eig_vals, eig_vecs = np.linalg.eig(R)
```

```
idx = np.argsort(eig_vals)[::-1]
```

```
eig_vals, eig_vecs = eig_vals[idx], eig_vecs[:, idx]
```

5. Inertie expliquée

```
inertie = eig_vals / np.sum(eig_vals)
```

6. Projection

```
F = np.dot(Z, eig_vecs)
```

7. Visualisations

- Éboulis des valeurs propres

- Cercle des corrélations

- Plan factoriel des individus

 **Bon courage pour votre TP !**